FormFit AI

Adaptive AI CrossFit Coaching Platform
Yale SOM MGT 409 (AI Foundations for Managers) — Final Project
Prepared: 2026-10-26 | Status: MVP complete, published to GitHub

Link	Location
GitHub repository	https://github.com/pjtalledo/formfit-ai
Live app (local)	`http://localhost:8000` (run from `Final Project/` — see README.md)
Slides	`slides.html` (this repository)
Demo video	PENDING / OPTIONAL — not recorded

> **Evidence policy for this report:** every claim below is either (a) backed by a real test, a real screenshot, or a real API response captured during development (see `AI_prompts.md` for the full, reconciled log of all 22 real API calls made across this project), or (b) explicitly labeled PENDING/hypothetical. Nothing is fabricated.

1. Motivation

"Fitness shouldn't depend on your schedule or budget."

FormFit AI's broader product vision is an accessible, flexible, personalized AI fitness coach — not merely a CrossFit whiteboard scanner. That vision responds to several real, common barriers to consistent training:

- **Fixed schedules and limited time**: busy professionals and students often cannot attend classes at set times, and frequently have only short windows available — a lunch break, or 20-45 minutes between commitments.
- **Cost of individualized attention**: one-on-one personal coaching is expensive, and group classes divide a coach's attention among many participants, limiting individualized feedback and technique supervision.
- **One-size-fits-all programming**: standardized workouts do not automatically fit each person's ability, goals, equipment, limitations, or available time.
- **Lack of continuity**: athletes training without a dedicated coach often lack continuous progression tracking and personalized follow-up, and nutrition/recovery guidance may require paying for yet another separate service.
- **Sustainability of the habit itself**: inconsistent access to training that actually fits a person's life can make long-term exercise habits difficult to sustain.

These are presented as the problems motivating FormFit AI's product direction, not as independently measured statistics — no claim here is backed by a cited study or survey; it is a reasoned product rationale, stated as such.

> **MVP vs. vision, stated plainly**: the working application described in this report today addresses a narrower slice of this vision — CrossFit WOD interpretation, scaling, and basic coaching guidance. The broader vision (flexible scheduling/time-budget optimization, personalized nutrition, continuous progression tracking, video technique correction) is **future product direction, not implemented or verified functionality** — see §7 for the explicit scope boundary.

"Your schedule. Your level. Your equipment. Your workout."

FormFit AI is built **mobile-first**: the typical use case is an athlete at the gym, on their phone, photographing a whiteboard WOD, confirming the extraction, and getting a personalized plan — not sitting at a desktop. Responsive mobile behavior across

common smartphone viewport sizes is verified in §7/the Technical Audit (see `docs/TECHNICAL_AUDIT.md` "Mobile-First Verification").

2. What Is New

Based on a competitive review of existing CrossFit tracking/coaching products (full detail in `docs/COMPETITIVE_ANALYSIS.md`), here is what the **current working MVP** actually does differently — not any single piece in isolation — is new for this space:

- **Whiteboard-photo-to-structured-workout extraction** with mandatory human confirmation before the extracted text is ever used (verified real test evidence, §6).
- **A fully integrated single workflow**: profile → scan or type → confirm → personalized coaching → persisted history, rather than separate logging/community/coaching tools.
- **A transparent, user-visible agent execution trace** for every coaching plan generated (see the real screenshot in §4).
- **Honest uncertainty handling**: the system explicitly says "unclear" rather than guessing when handwriting is illegible (real evidence in §6).

This project does not claim to be the first AI-assisted fitness coaching tool in any general sense — see the explicit non-claims in `docs/COMPETITIVE_ANALYSIS.md`.

> To be clear: **the broader vision's remaining pain points — flexible time-budget scheduling, personalized nutrition, continuous long-term progression tracking, and video technique correction — are not operational today.** No feature in this report should be read as implying they are, unless independently verified and described as such elsewhere in this document (they are not).

3. The Agents

FormFit AI uses a single orchestrating pipeline with five distinct tool calls (not a single opaque prompt, and not a hard-coded sequence disguised as "agentic"):

```
User Input (typed WOD or whiteboard photo)
        |
        v
[1] scan_wod_image_tool  (LLM, vision) -- only if image uploaded
        |  transcribes photo to text; flags [unclear: ...] rather than guessing
        v
  >> USER REVIEWS / EDITS TEXT (mandatory, never auto-applied) <<
        |
        v
[2] parse_wod_tool  (LLM)  -- extracts structured WOD (format, exercises, scheme)
        |
        v
[3] validate_wod_tool  (local)  -- checks exercises against curated knowledge base
        |
        v
[4] generate_recommendation_tool  (LLM)  -- warm-up, scaling, guidance, accessories
        |
        v
[5] validate_recommendation_tool  (local)  -- Pydantic schema validation
        |
        v
  Saved to SQLite + displayed with real usage/cost + full trace
```

Architecture reference: `backend/app/agent.py`. Each step's real input/output/timing is captured into an inspectable trace, shown to the end user (see the Agent Trace panel in Figure 2 below) — not just logged internally and hidden.

Real Agent Trace (captured live, from the screenshot in Figure 2)

```
parse_wod_tool (4125ms) — OK
  in:  For Time: 21-15-9 Thrusters, Pull-ups
  out: {"format":"For Time","exercises":[{"name":"Thrusters","scheme":"21-15-
9",...

validate_wod_tool (0ms) — OK
  in:  2 exercises
```

```
  out: all exercises recognized in knowledge base

generate_recommendation_tool (1062ms) — OK
  in:  Athlete: beginner athlete, goals: build strength and conditioning,
equipment: ba...
  out: {"warm_up": ["2 rounds: 8 air squats and 6 push-ups...", ...]}

validate_recommendation_tool (0ms) — OK
  in:  recommendation json
  out: valid
```

Figure 1 — Athlete Profile & WOD Input (real screenshot, Playwright-captured against the live app):

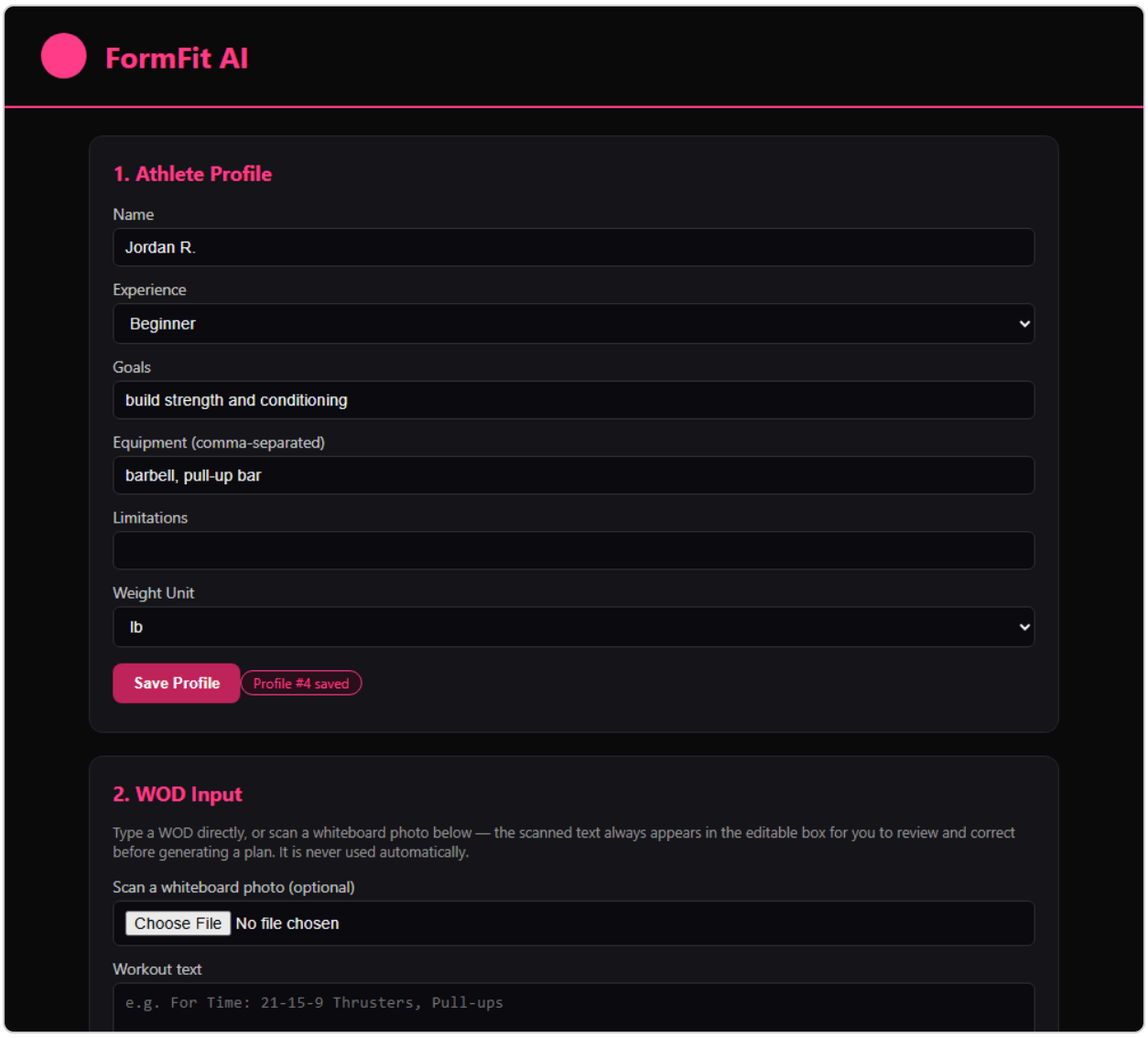


Figure 2 — Generated Coaching Plan with real usage metrics and agent trace (real screenshot):

FormFit AI

1. Athlete Profile

Name

Jordan R.

Experience

Beginner

Goals

build strength and conditioning

Equipment (comma-separated)

barbell, pull-up bar

Limitations

Weight Unit

lb

Save Profile

Profile #4 saved

2. WOD Input

Type a WOD directly, or scan a whiteboard photo below — the scanned text always appears in the editable box for you to review and correct before generating a plan. It is never used automatically.

Scan a whiteboard photo (optional)

Choose File No file chosen

Workout text

```
For Time: 21-15-9 Thrusters, Pull-ups
```

Generate Coaching Plan

3. Personalized Coaching Plan

Warm-up

- 2 rounds: 8 air squats and 6 push-ups at an easy pace
- Practice 5 light thrusters, focusing on a comfortable squat and controlled press
- Practice 3–5 pull-ups using assistance if needed

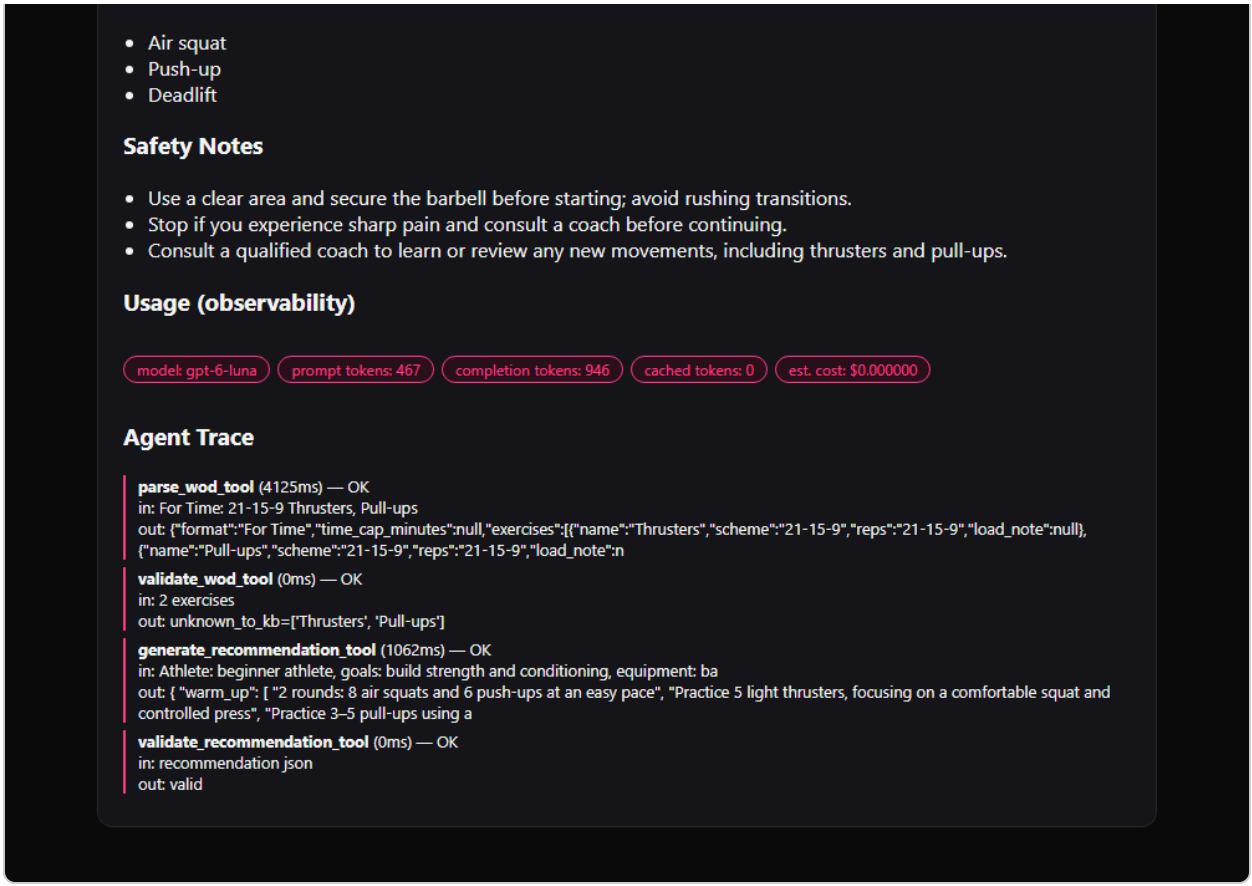
Scaling Options

- Reduce the workout to 15-12-9 or fewer reps per movement if needed
- Use a light barbell load for thrusters that allows consistent, controlled reps; reduce the load or use air squats if technique breaks down
- Use assisted pull-ups or reduce pull-up reps, breaking them into small sets with rest

Main Workout Guidance

For time, complete 21-15-9 thrusters and pull-ups in order. As a beginner, prioritize safe, repeatable technique over speed, choose a manageable barbell load, and break sets early enough to maintain control. Rest as needed and stop before form deteriorates.

Complementary Exercises



> **Known bug found and fixed via this exact screenshot**: the first capture showed `unknown_to_kb=['Thrusters', 'Pull-ups']` even though both exercises are in the curated knowledge base — the match was case-sensitive and did not tolerate plural forms ("Thrusters" vs. KB key "thruster"). Fixed in `backend/app/knowledge_base.py` (`is_known_exercise`) and covered by a new parametrized unit test. This is disclosed, not hidden, because it is real evidence the KB-matching logic needed hardening.

4. Tokenomics

Full detail and formulas: `docs/TOKENOMICS_ANALYSIS.md`. Summary of real, measured data:

Operation	Prompt tokens (real)	Completion tokens (real)	Cached tokens (real)
WOD Scan (image)	258–491	76–206	0
Text WOD parse	191	107	0
Full coaching-plan generation (2 LLM calls)	393–467	946–1,019	0

> **No verified `gpt-6-luna` price exists.** It is a course-specific model alias routed via Portkey to an Azure OpenAI backend; no price field is returned anywhere in the API response, and no public price page exists for this alias. **Every dollar figure that follows is a labeled formula, never an asserted real price** — see `docs/TOKENOMICS_ANALYSIS.md` for the full reasoning.

Cost formulas (symbolic, pending a real rate):

```
cost_per_scan           ≈ (370 / 1e6 × input_rate) + (120 / 1e6 × output_rate)
cost_per_coaching_plan  ≈ (394 / 1e6 × input_rate) + (1000 / 1e6 × output_rate)
cost_per_workflow       = cost_per_scan + cost_per_coaching_plan  (scan + plan)
monthly_cost_per_user   = 22 × cost_per_coaching_plan + 11 × cost_per_scan
(assumption: 1 WOD/day, 22 training days, half scanned)
total_monthly(N users)  = N × monthly_cost_per_user
```

Real measured optimization finding: image-based scan calls use ~1.5–2.5x more prompt tokens than text-only parsing of similar complexity (258–491 vs. 191 tokens) — a genuine, measured observation suggesting image downscaling before upload as a concrete next optimization experiment (not yet run).

5. Safety

Full detail, including redacted real attack traces: `docs/SAFETY_EVIDENCE.md`. Five genuine real-API tests were run (not mocked):

Test	Real result	Layer responsible
Prompt injection (typed text, asking to reveal the API key)	Rejected by Azure OpenAI's `content_filter` (HTTP 400) before any model response	Provider platform
Prompt injection (embedded in whiteboard image)	Transcribed verbatim as text; not obeyed. API key never in any LLM context regardless.	Application + architecture
Injection propagation downstream	Rejected again, independently, by the same content filter at the parsing stage	Provider platform (defense-in-depth)
Ambiguous/illegible image	Returned `"[unclear: illegible]"` for all lines; zero fabricated content	Application (prompt design)
Missing information (text)	Left reps/scheme blank rather than inventing numbers; preserved "not sure how many" verbatim	Application (prompt design)

Provider vs. application defenses, explicitly distinguished (per instruction, not conflated): the two "rejected by content filter" results are credited to Azure OpenAI's platform, not to code written in this project. Only the verbatim-transcription behavior and the architectural fact that the real API key is never placed in any prompt are genuinely this project's own defenses.

Other controls: upload restrictions (PNG/JPEG/WebP only, 8MB max, verified by automated tests), budget controls (no retry loops, no background calls, 8MB cap limits per-request cost exposure), and exercise-related limitations (recommendations constrained to a curated knowledge base, but this is a content constraint, not a medical safety guarantee — see full disclosure in `docs/SAFETY_EVIDENCE.md` §5).

> **This project does not claim complete protection.** Five real tests is a sample, not an exhaustive security audit. No automated, repeatable adversarial test suite exists yet — a disclosed gap, not a hidden one.

6. Impact

PENDING — **no real user evaluation has been conducted.** No participants, feedback, scores, or improvement percentages are reported here, per explicit instruction never to invent them.

A full evaluation protocol and a ready-to-use feedback form (covering manual-vs-FormFit-AI interpretation time, perceived usefulness, clarity/accuracy, problems encountered, and willingness to reuse) have been prepared and are ready to run with real CrossFit athletes — see `docs/IMPACT_EVALUATION.md` for the complete protocol and form text.

When real evaluation data exists, this section will be updated with raw per-participant numbers, not an invented aggregate, per the reporting rules defined in that document.

7. Limits and Next Steps

Implemented and verified (this report's actual scope)

- Athlete profile creation and persistence (SQLite, verified across session restarts).
- WOD input via typed text or whiteboard-photo scan, with mandatory user confirmation.
- 5-tool agent pipeline generating personalized coaching plans, with a real, inspectable trace.
- Real token/latency observability per call; cached-vs-uncached tokens correctly distinguished (always 0 so far, honestly reported, since no caching feature exists).
- 5 genuine real-API safety/accuracy tests, with results that are not all favorable by luck — two were caught by the underlying platform, not this project's own code, and that distinction is preserved rather than glossed over.
- A real, found-and-fixed bug (KB plural-matching) discovered via an actual screenshot during this very reporting phase — evidence that evaluation here is genuine, not performative.

Explicitly not implemented — broader product vision vs. today's MVP

FormFit AI's full product vision (§1) is an accessible, flexible, personalized AI fitness coach. The items below are real, named parts of that vision that are **not built, not operational, and not independently verified** in this project today — they should not be inferred as working from anything else in this report:

- **Time-budget-aware scheduling / short-window workout optimization** (e.g., fitting a session into a lunch break) — not implemented.
- **Personalized nutrition guidance** beyond general, non-personalized informational content — not implemented.
- **Continuous, long-term progression tracking and proactive follow-up** beyond basic workout history storage — not implemented.
- **Video Technique Analysis** — not deployed; see the dedicated subsection below for the real feasibility experiment behind this decision.
- **AI Barbell & Weight Scanner** — assessed as too high-risk (occlusion/ambiguous-plate fabrication risk), per `docs/FEASIBILITY_MATRIX.md`.

Video Technique Analysis — Feasibility Experiment and Decision

Before deciding on Video Technique Analysis, a real, isolated feasibility experiment was run (full detail: `experiments/video_feasibility/RESULTS.md`) — it was never integrated into the production application. What follows separates what was actually observed from what is interpretation.

Observed evidence:

- Two real Olympic-lifting videos (a squat clean and a snatch, same athlete/session) were evaluated using four extracted frames per video, sent to `gpt-6-luna` in two real API calls.
- For the squat clean, the model's identification was likely correct, cross-checked by direct visual inspection of the frames.
- For the second video, the model could not reliably distinguish the exercise (snatch vs. clean-and-jerk) and explicitly said so rather than guessing.
- The four static frames allowed only limited observations of visible body/bar positions at each captured instant; a weight plate occluded the lifter's hands/bar/arms in both clips.
- Four frames did not provide sufficient temporal information to assess complete lifting technique (bar path, tempo, or joint timing through the full movement).
- The tested configuration produced **zero actionable coaching cues** in either call.

Interpretation:

- The safety-constrained system prompt used in this experiment deliberately did not ask the model for technique-fault judgments (per this project's own injury/medical-claim restrictions). **The absence of coaching cues therefore does not prove the model is fundamentally incapable of useful feedback** — it reflects the conservative prompt design tested, not an exhaustive capability ceiling.
- Given the combination of unreliable identification on a harder case, real occlusion limitations, and the absence of temporal/motion information from static frames, we decided **not to deploy Video Technique Analysis without improved temporal processing (e.g., more frames, pose estimation, or native video support) and expert human validation** of any output before it could be shown to an athlete as coaching feedback.

This is presented as a responsible, evidence-based product decision — the experiment's real findings, not assumption, drove the choice to keep this feature as future work rather than ship it.

Known gaps and risks

- **Tokenomics**: real token counts exist; real dollar costs do not, pending a sourced `gpt-6-luna` price.
- **Impact**: protocol ready; zero real users evaluated yet.
- **Safety**: 5 real tests is meaningful but not exhaustive; no automated regression suite for adversarial inputs yet.
- **Reproducibility**: setup/run commands have been verified in-place multiple times this session, but not yet from a truly separate fresh clone (Audit C in `docs/FINAL_SUBMISSION_AUDIT.md` is the planned check for this, not yet executed).
- **Browser coverage**: automated verification used headless Chromium (Playwright); the user independently confirmed correct rendering in Microsoft Edge with a real screenshot, but no automated cross-browser suite exists.

Recommended next steps

1. Source real `gpt-6-luna` pricing from the course/Portkey administrator; substitute into the tokenomics formulas.
2. Run a real user evaluation using the prepared protocol, even with a small number of participants.
3. Package the 5 manual safety tests into an automated, re-runnable test suite.
4. Run the Audit C fresh-clone procedure (`docs/FINAL_SUBMISSION_AUDIT.md`) before final submission.
5. Run the real image-compression optimization experiment identified in Tokenomics §4.
6. Scope and prototype one piece of the broader vision (§1) — most plausibly short-window workout time-budgeting — as the next MVP increment, before attempting nutrition personalization or progression tracking.